

1 Problem 5

1.1 Part b

The code for this problem can be found in § 2.

To implement the code for part b, I defined a function ‘elliptical_solve_1d’ that takes a function ‘f’ and a number of grid-points ‘n’. This function discretized the interval $(0, 1)$ into $n - 1$ points denoted ‘x’, computes the b vector as $f(x)$, creates a tri-diagonal matrix ‘A’ that follows the specifications described in part a, and uses ‘`scipy.sparse.linalg.spsolve`’ to solve the system $Au = b$. This function returns ‘u’ and ‘x’.

The rest of the code I wrote for this problem are intermediate and plotting functions. There is nothing special about how I set up my ‘`matplotlib.pyplot`’ plots, so please see the Appendix (§ 2) for more detail.

Below are the plots for $k=3, 5, 10$. From the $k=3$ case, we see the grid is far too coarse to provide a good approximation. The $k=3$ case misses the local minima and maxima and shows, in a sense, a more averaged function. For the $k=5$ case, we already see how much better of an approximation quadrupling the number of grid points provides. I specifically chose $k=5$ because it was the first k where I would visually say "that approximation looks pretty close to the exact solution. All the extrema are captured and $k=5$ provides a reasonable approximation of the exact function. For $k=10$, the approximate and exact functions are visually identical and a human could not tell you the difference between the two.

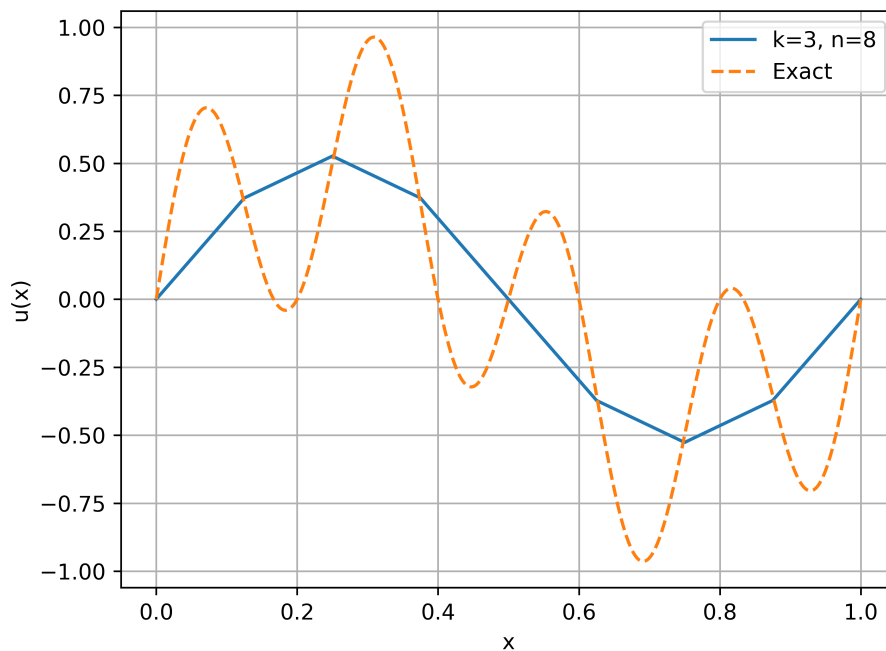


Figure 1: Plot of Approximate $u(x)$ and exact $u(x)$ for $k=3, n=8$.

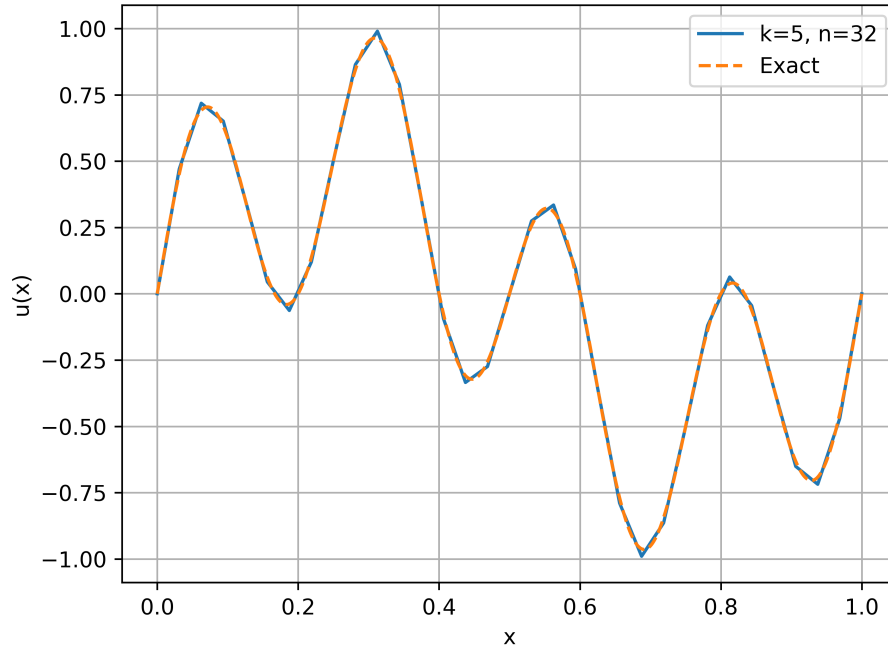


Figure 2: Plot of Approximate $u(x)$ and exact $u(x)$ for $k=5$, $n=32$.

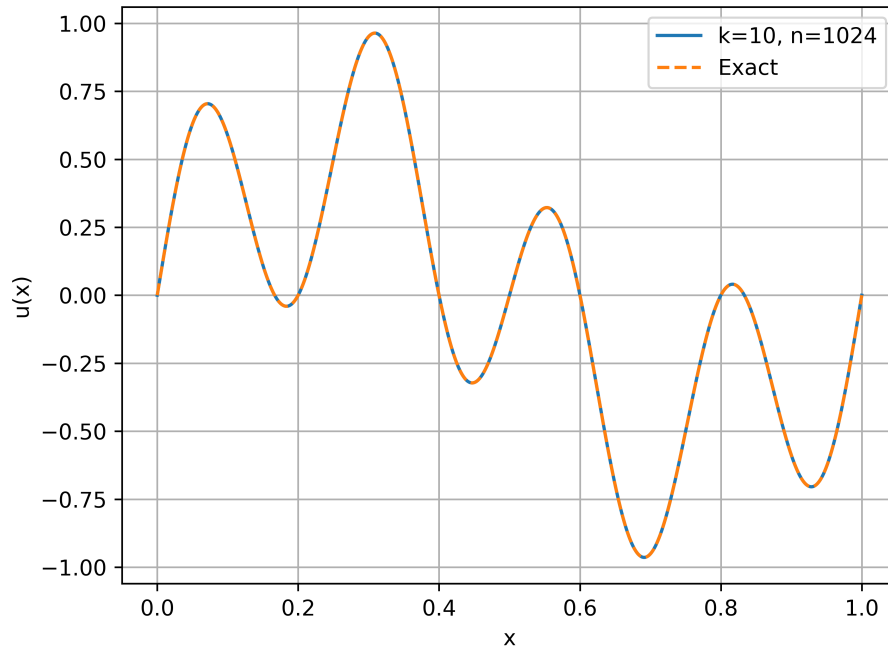


Figure 3: Plot of Approximate $u(x)$ and exact $u(x)$ for $k=10$, $n=1024$.

1.2 Part C

Table 1: Error analysis and convergence ratios.

k	n	$E_k = \ \mathbf{u}_k - \mathbf{u}_{\text{exact}}\ _\infty$	Ratio (E_{k-1}/E_k)
3	8	2.65146e-02	—
4	16	1.22833e-01	0.21586
5	32	2.80016e-02	4.38663
6	64	6.84658e-03	4.08987
7	128	1.70225e-03	4.02208
8	256	4.24979e-04	4.00549
9	512	1.06208e-04	4.00137
10	1024	2.65498e-05	4.00034

When investigating the error ratios, we see the asymptotic error ratio approaches four for every doubling of n . Doubling the number of points in the spatial discretization scheme causes a four-fold reduction in the error. From this, we see this numerical method has a second-order or $O(h^2)$ convergence rate.

2 Appendix

Listing 1: Question 5 code

```
import matplotlib.pyplot as plt
import numpy as np
from scipy.sparse.linalg import spsolve
from scipy.sparse import diags

# PART B #####
def elliptic_solve_1d(f, n):
    x = np.linspace(0, 1, n + 1)[1:-1]
    b = f(x)
```

```

h = 1 / n
main_diag    = 2 / h**2 * np.ones(n - 1)
sub_diag     = -1 / h**2 * np.ones(n - 2)
super_diag   = -1 / h**2 * np.ones(n - 2)

# format of csr stands for Compressed Sparse Row
A = diags([main_diag, sub_diag, super_diag], [0, -1, 1], format="csr")
u = spsolve(A, b)

return u, x

def f_partB(x):
    t1 = 17 * np.cos(3 * np.pi * x) * np.sin(5 * np.pi * x)
    t2 = 15 * np.sin(3 * np.pi * x) * np.cos(5 * np.pi * x)
    return 2 * np.pi**2 * (t1 + t2)

exact_solution_partB = lambda x: np.cos(3*np.pi*x) * np.sin(5*np.pi*x)

# plotting
def plot_total_partB():
    ks = np.arange(3, 10 + 1)
    ns = 2**ks

    for n in ns:
        u, x = elliptic_solve_1d(f_partB, n)

        x_full = np.concatenate(([0], x, [1]))
        u_full = np.concatenate(([0], u, [0]))

```

```

plt.plot(x_full, u_full, label=f"n={n}")

exact_x = np.linspace(0, 1, 10_000 + 1)
exact_u = exact_solution_partB(exact_x)
plt.plot(exact_x, exact_u, label="Exact")

plt.legend()
plt.show()

def plot_single_partB(k, show):
    n = 2**k

    # approximate

    u, x = elliptic_solve_1d(f_partB, n)
    x_full = np.concatenate(([0], x, [1]))
    u_full = np.concatenate(([0], u, [0]))

    plt.plot(x_full, u_full, label=f"k={k}, n={n}")

    # analytical

    exact_x = np.linspace(0, 1, 10_000 + 1)
    exact_u = exact_solution_partB(exact_x)
    plt.plot(exact_x, exact_u, label="Exact", ls="—")

    # plot parameters

    plt.xlabel("x")
    plt.ylabel("u(x)")

    plt.grid(which="both")
    plt.legend()

```

```

plt.savefig(f"hw01_partB_n{n}", dpi=600)
if show: plt.show()
plt.close()
return

# plotting
def do_partB(show):
    plot_single_partB(3, show)
    plot_single_partB(5, show)
    plot_single_partB(10, show)

do_partB(show=False)

#####

# PART C #####
ks = np.arange(3, 10 + 1)
ns = 2**ks

error_ks = []

for n in ns:
    u_approx, x = elliptic_solve_1d(f_partB, n)
    u_exact = exact_solution_partB(x)

    error_norm = np.linalg.norm(u_approx - u_exact, np.inf)
    error_ks.append(error_norm)

error_array = np.array(error_ks)
ratios = error_array[:-1] / error_array[1:]

```

```

# latex table

latex_table = "\\begin{table}[h]\\n\\centering\\n"
latex_table += "\\begin{tabular}{cccc}\\n\\hline\\n"
latex_table += "_$k$_&$_n$_&$_E_k$=\\n\\boldsymbol{u}_k-\\n\\boldsymbol{u}_{\\text{exact}}\\n"

# Loop to populate rows
for i, (k, n, err) in enumerate(zip(ks, ns, error_ks)):
    if i == 0:
        # The first entry (k=3) has no previous error, so ratio is N/A
        latex_table += f"_{k}_&_{n}_&_{err:.5e}&_—_\\n\\n"
    else:
        # Use your ratios array (index shifted by 1 since ratios starts at k=4)
        latex_table += f"_{k}_&_{n}_&_{err:.5e}&_{ratios[i-1]:.5f}_\\n\\n"

latex_table += "\\hline\\n\\end{tabular}\\n"
latex_table += "\\caption{Error_analysis_and_convergence_ratios.}\\n"
latex_table += "\\label{tab:error_convergence}\\n\\end{table}"

print(latex_table)

#####

```